

RoadGuard

Complete Project Guide

Smart Pothole Detection & Reporting System

Generated 8/13/2026

RoadGuard — Complete Project Guide

Smart Pothole Detection & Reporting System

This document explains the project in detail: purpose, technology, every algorithm, process flows, methods, and known limitations. It is written for developers, reviewers, and stakeholders who need a single reference for how RoadGuard works end to end.

Table of Contents

1. Project Overview 2. Technology Stack 3. System Architecture 4. User Roles & Permissions 5. End-to-End Process Flows 6. Algorithms (Detailed) 7. Supporting Methods & Pipelines 8. Database Design 9. Frontend Modules 10. Backend Modules 11. Security & Quality Methods 12. Deployment & Operations 13. Limitations & Future Work

1. Project Overview

1.1 Purpose

RoadGuard is a web-based government-style platform that lets citizens report potholes using photos and GPS location, while giving municipal administrators tools to verify, prioritize, assign field workers, route crews, and track repairs.

1.2 Core Capabilities

Stakeholder	What they can do
Public (no login)	View landing page, browse public pothole map, submit contact form
Citizen (USER)	Register/login, submit reports with AI gate, track status,...
Administrator (ADMIN)	Full report workflow, priority queue, worker assignment, Dijkstra...

1.3 Design Principles

- Monorepo — one repository, npm workspaces (frontend + backend)
 - TypeScript everywhere — compile-time safety on client and server
 - MVC + Repository pattern on the backend — routes !' controllers !' services !' repositories !' Prisma
 - Algorithm module — pothole detection, duplicates, priority, routing live in backend/src/algorithms/
 - REST API — JWT access token + rotating refresh token
-

2. Technology Stack

2.1 Frontend

Tool	Version / Role	Why it is used
React 18	UI library	Component-based SPA
TypeScript	Static typing	Fewer runtime errors, better IDE support
Vite	Build tool & dev server	Fast HMR, modern ES modules
Tailwind CSS	Utility-first styling	Consistent “government dashboard” design
React Router v6	Client routing	Protected routes for user/admin
TanStack React Query	Server state	Caching, refetch, loading states for API data
React Hook Form + Zod	Forms & validation	Performant forms with schema validation
Axios	HTTP client	Interceptors for JWT + token refresh
Framer Motion	Animations	Page transitions, drawers
Leaflet + react-leaflet	Maps	Public map, location picker, admin route map
leaflet.heat	Heatmap layer	Admin geographic density
Chart.js + react-chartjs-2	Charts	Admin analytics (bar, pie, line)
html2canvas	Map capture	Screenshot-based map report submission
Vitest + Testing Library	Tests	Component and guard tests

2.2 Backend

Tool	Role	Why it is used
Node.js "e 22	Runtime	Modern JS, native fetch
Express	HTTP framework	REST API, middleware pipeline
TypeScript	Static typing	Shared types with frontend concepts
Prisma	ORM	Type-safe DB access, migrations
PostgreSQL (Supabase)	Production database	Scalable relational storage
bcryptjs	Password hashing	Secure credential storage
jsonwebtoken	JWT auth	Stateless access + refresh sessions
Zod	Request validation	Body/query schema validation
Multer	Multipart uploads	Photo and avatar file handling
sharp	Image processing	Resize, WebP conversion, annotation
Cloudinary (optional)	Cloud storage	Production image hosting; local /uploads...
pdfkit	PDF generation	Report receipts on server
exceljs	Excel export	Admin data export
helmet	Security headers	HTTP hardening
cors	Cross-origin control	Allowlist frontend origin
express-rate-limit	Rate limiting	Brute-force protection on auth
google-auth-library	OAuth	Google Sign-In verification
Vitest + Supertest	Tests	Unit + API integration tests

2.3 Infrastructure

Component	Typical setup
Frontend hosting	Vercel (static SPA + proxy to API)
Backend hosting	Railway / Render / Node server
Database	Supabase PostgreSQL
Images	Cloudinary or local backend/uploads/

3. System Architecture

3.1 High-Level Diagram

Flow diagram (see docs/project-guide.md for Mermaid source)

3.2 Request Flow

```
Browser !' React Router !' Page component
  !' React Query (api client)
    !' Axios (+ JWT interceptor)
      !' Express route
        !' authenticateToken / requireRole
        !' validateBody (Zod)
        !' upload.single (Multer, if multipart)
          !' Controller (thin HTTP layer)
            !' Service (business logic + algorithms)
              !' Repository (Prisma queries)
                !' PostgreSQL
```

Errors return a consistent JSON envelope: { success: false, error: { message, code?, fields? } }.

3.3 Folder Structure (Summary)

```
roadguard-react/
% % % frontend/src/      pages, components, api, features/auth, router, lib
% % % backend/src/
% % % % algorithms/     CNN, Dijkstra, heap, geo, duplicate, image, routing
% % % % controllers/    HTTP handlers
% % % % services/       business logic
% % % % repositories/   data access
% % % % routes/         Express routers
% % % % middleware/     auth, upload, validate, rateLimit, error
% % % % utils/          PDF export, tokens, ApiError
% % % backend/prisma/    schema, migrations, seed
% % % docs/              SRS, API, algorithms, this guide
```

4. User Roles & Permissions

Role	Access
Public	Landing, public map, contact form
USER	Dashboard, new report, my reports, profile, notifications, receipt...
ADMIN	Admin dashboard, verify/reject/assign reports, priority queue,...

Authentication flow:

1. Register or login !' server returns accessToken, refreshToken, (Authorization: Bearer ...)
3. On 401, frontend tries one refresh; if refresh token server-side

5. End-to-End Process Flows

5.1 Citizen Report Submission

Flow diagram (see docs/project-guide.md for Mermaid source)

Steps in plain language:

1. Citizen uploads or captures a road photo. 2. Pre-submit AI gate (POST /reports/detect) runs detection and returns confidence, severity, bounding box, annotated preview. 3. Citizen pins GPS on map (or uses browser geolocation / reverse geocode). 4. On submit, server re-runs detection — submission without a pothole can be rejected. 5. Duplicate detection finds open reports within 20 m; user may continue or cancel. 6. Report saved as PENDING with CNN severity, priority score, image hash, timeline entry. 7. Admins receive notifications.

5.2 Admin Report Workflow

Flow diagram (see docs/project-guide.md for Mermaid source)

Admin actions:

- Verify — confirms report is valid; may use Verify-AI to accept/reject CNN result
- Assign — pick field worker; system can auto-pick nearest by driving ETA (Dijkstra)
- Update status — move through workflow; add remarks; upload completion image
- Priority queue — process-next pops highest priority open report and assigns route
- Export — PDF receipt, CSV, Excel for stakeholders

5.3 Priority !' Routing !' Assignment Flow

Flow diagram (see docs/project-guide.md for Mermaid source)

5.4 Algorithm Pipeline (Big Picture)

Photo + GPS	
%	
%¼	
% % % % % % % % % % % % severity, confidence, bbox	
% CNN + scene % ° priority score	
% + structure %	
% % % % % % % % % % % %	
% GPS	
%¼	
% % % % % % % % % % % % confirmations++ on duplicate	
% Haversine %	
% duplicate %	
% % % % % % % % % % % %	
%	
%¼	
% % % % % % % % % % % % highest score first	
% Max Heap %	
% priority Q %	
% % % % % % % % % % % %	
% process-next	
%¼	
% % % % % % % % % % % % polyline + ETA on map	
% Dijkstra %	
% routing %	
% % % % % % % % % % % %	

6. Algorithms (Detailed)

6.1 CNN — Pothole Detection & Severity Classification

Location: backend/src/algorithms/cnn/ Weights: backend/data/cnn-weights.json (trained offline) Entry point: backend/src/algorithms/cnn/detector.ts

What it does

Classifies a road photo into five classes:

Index	Class	Meaning
0	NONE	No pothole
1	LOW	Minor damage
2	MEDIUM	Moderate damage
3	HIGH	Serious damage
4	CRITICAL	Severe hazard

The CNN is implemented from scratch in TypeScript, using TensorFlow.js, Keras.js, and the tfjs-backend-cpu optimizer, softmax — ~21k parameters.

Architecture (32×32 grayscale input)

```
Input 32×32×1
! ' Conv1 (32 filters, 3×3, SAME) + ReLU ! ' MaxPool 2×2
! ' Conv2 (48 filters, 3×3, SAME) + ReLU ! ' MaxPool 2×2
! ' Dual pooling head:
    • per-channel mean (96 values)
    • per-channel top-3 mean
    • input depth-floor (mean of 3 darkest pixels) — recovers signal lost through ReLU+max-pool
! ' Dense 96! '64 + ReLU
! ' Dense 64! '5
! ' Softmax ! ' class probabilities
```

Inference pipeline

1. Resize photo to 32×32 grayscale; normalize to [0, 1] 2. Run forward pass ! ' softmax probabilities 3. Road scene check (roadScene.ts) — reject portraits, selfies, sky/grass-dominated frames (RGB + YCbCr skin detection) 4. Structure check (potholeStructure.ts) — confirm dark hole-shaped region on asphalt 5. Verdict logic (evaluateCnnVerdict) — combines CNN + geometry; blocks false positives from camera/face photos 6. Bounding box — class activation map (CAM) projected to normalized { x, y, width, height } 7. Severity — CNN argmax (excluding NONE) is authoritative; citizen form choice stored as suggestedSeverity

Training

- Script: backend/scripts/train-cnn.ts
- Synthetic dataset — non-overlapping severity bands by blob radius/depth on gray road background
- Optimizer: Adam (lr 0.01), Gaussian soft-target cross-entropy
- Target: "e 95% validation accuracy before weights are committed"
- Retrain: cd backend && npx tsx scripts/train-cnn.ts

Complexity

- Inference: $O(C \cdot K^2 \cdot H \cdot W)$ per layer — microseconds on CPU
- Limitation: trained on synthetic data, not real-world labeled pothole photos (see §13)

6.2 Heuristic Detector (Fallback / Tests)

Location: backend/src/algorithms/heuristicDetector.ts Activation: ROADGUARD_DETECTOR=heuristic
environment variable

Method

1. Downscale to 32×32 grayscale grid 2. Compute median luminance as “road baseline” 3. Threshold dark cells (< median × 0.75) 4. Find largest connected dark cluster (flood fill) 5. Score by cluster size + contrast !' confidence + bounding box

Purpose

- Lightweight stand-in with no weight file
- Used in unit tests and development
- Not intended as production-grade detection

6.3 Road Scene Analysis

Location: backend/src/algorithms/roadScene.ts

Method

Downscale to 64×64 RGB sample; classify pixels:

Check	Rule
Skin	RGB ratio rules + YCbCr skin cluster (catches selfies/webcam)
Road	Low saturation, gray asphalt/concrete tone
Green	Vegetation dominance
	If skinRatio too high or roadRatio too low !' not road-like !' reject before

6.4 Pothole Structure Analysis

Location: backend/src/algorithms/potholeStructure.ts

Method

Analyzes grayscale structure for a dark, compact region on a light background, reacting to random dark objects. Used as a geometry gate alongside

6.5 Haversine Formula — Distance on Earth

Location: backend/src/algorithms/geo.ts

Formula

Great-circle distance between two (latitude, longitude) points in meters

```
a = sin²(Δφ/2) + cos φ₁ · cos φ₂ · sin²(Δλ/2)
c = 2 · atan2(√a, √(1−a))
distance = R · c      (R = 6,371,000 m)
```

Three uses in RoadGuard

Use	Description
Duplicate detection	Find open reports within 20 m (DUPLICATE_RADIUS_M)
Graph snapping	Map report/worker GPS to nearest OSM road node
Reference distance	Straight-line km shown alongside driving route
Complexity: O(1) per pair; duplicate scan is O(reports in radius).	

6.6 Duplicate Detection

Location: backend/src/algorithms/duplicate.ts, report.repo.findOpenN

Rules

- Radius: 20 metres (Haversine)
- Only open statuses count: PENDING, VERIFIED, ASSIGNED, IN_PROGRESS
- On duplicate: increment confirmations on existing report, recompute priority score
- Optional image hash (perceptual hash + Hamming distance) for similar-photo detection

User experience

- Pre-submit: POST /reports/check-duplicate or duplicate response on create (409)
- User can cancel or continue (if policy allows)

6.7 Priority Score & Max Heap

Location: backend/src/algorithms/priority.ts, heap.ts, priorityQueue.service.ts

Score formula

```
priorityScore =
  SEVERITY_WEIGHT[severity]    // LOW=10, MEDIUM=20, HIGH=30, CRITICAL=40
+ 12 × min(confirmations, 4)  // citizen duplicate confirmations, capped
+ min(ageDays, 20)            // older reports gain urgency
+ min(traffic, 10)            // optional traffic factor 0..10
```

Max heap

- BinaryHeap<T> with custom comparator — max-heap for priority queue
- Same class used as min-heap for Dijkstra (opposite comparator)
- Rebuilt on demand from open reports — O(n) heapify
- process-next: pop highest score !' plan route !' assign worker !' status ASSIGNED

Example

- Fresh CRITICAL (no confirmations): 40 + 0 + 0 + 0 = 40
- HIGH with 3 confirmations: 30 + 36 + age + traffic = 66+ !' beats fresh CRITICAL

6.8 Dijkstra's Algorithm — Shortest Driving Route

Location: backend/src/algorithms/dijkstra.ts, roadGraph.ts, routing.ts

Road graph

- Built from OpenStreetMap via Overpass API
- Script: backend/scripts/fetch-road-network.ts
- Cached: backend/data/road-graph.json (~173k nodes, ~360k edges, Kathmandu valley)
- Each edge: distanceM (Haversine) + seconds (distance / road-class speed)

Road class	Speed (km/h) example
motorway	90
primary	55
secondary	45
tertiary	35
residential	25

Algorithm

- Min-heap frontier keyed by accumulated travel seconds
- Each node settled once; predecessor chain reconstructs path
- Complexity: $O(E \cdot \log V)$

Routing steps (planRoute)

1. Snap crew position and pothole GPS to nearest graph nodes (Haversine) 2. If either snap > 2 km from network !' reachable: false, reason off-network 3. Run Dijkstra; if no path !' no-route 4. Return { distanceKm, etaMinutes, path: [lat,lng][] } for map polyline

Team selection

nearestTeamByRoad — for each positioned worker, compute driving ETA; pick lowest ETA, not straight-line distance.

6.9 Image Processing

Location: backend/src/algorithms/image.ts

Function	Purpose
validateImageFile	MIME type, size "d 5 MB
processImage	Rotate (EXIF), resize max 1200px, WebP quality 80, blank-image...
processAvatarImage	400×400 cover crop for profile photos
computeImageHash	16×16 grayscale average hash for duplicate photos
hammingDistance	Compare two hashes
storeImage	Cloudinary upload or local uploads/<uuid>.webp

6.10 Perceptual Image Hash (Auxiliary)

Used to detect visually similar duplicate uploads:

1. Grayscale 16×16 resize
2. Compare each pixel to mean !' binary same image

7. Supporting Methods & Pipelines

7.1 Detection Service

File: backend/src/services/detection.service.ts

```
Upload file
! ' validateImageFile
! ' processImage (sharp)
! ' resolveDetector() ! ' cnnDetector (default) or heuristicDetector
! ' annotateBox (SVG rectangle via sharp composite)
! ' storeImage (preview URL)
```

Used by:

- POST /api/reports/detect — form step-2 gate
- report.service.create — mandatory re-detection on submit

7.2 Report Creation Methods

File: backend/src/services/report.service.ts

- Validates input, runs detection, stores images
- Computes priorityScore from CNN severity
- Writes StatusHistory, notifies admins
- Handles duplicate 409 response

7.3 Receipt PDF Generation

File: backend/src/utils/exporters.ts

- Server-side PDF with report metadata, photos, timeline
- Dynamic row layout, page breaks, embedded images from /uploads or Cloudinary

7.4 Auth Methods

File: backend/src/services/auth.service.ts

- bcrypt password hashing (cost 12)
- JWT access (short) + refresh (long, hashed in DB)
- Refresh token rotation + reuse detection (revokes session)
- Google OAuth token verification
- Profile update with optional avatar (multipart)
- Soft account deletion (anonymize, preserve reports)

8. Database Design

ORM: Prisma Production: PostgreSQL (Supabase) Schema: backend/prisma/schema.prisma

Core entities

Table	Purpose
User	Accounts, roles, worker flag, coords, avatarUrl
Report	Pothole reports + AI fields + priority + status
Location	Normalized geo/address (1:1 with report)
StatusHistory	Audit trail of status changes
Assignment	Worker assignment records
Notification	In-app user notifications
AdminLog	Admin action audit
ContactMessage	Public contact form

Key report fields (AI & workflow)

Field	Meaning
severity	Authoritative severity (from CNN on create)
suggestedSeverity	Citizen's form choice
confidenceScore	Detection confidence 0..1
boundingBox	JSON normalized detection box
detectedImageUrl	Annotated preview image
aiSeverity, aiClassProbs	Raw CNN output
aiVerified, aiRejectedReason	Admin Verify-AI verdict
confirmations	Duplicate confirmation count
priorityScore	Heap ordering key
completionImageUrl	After-repair photo
See database.md for full column lists.	

9. Frontend Modules

Module	Path	Responsibility
Landing	pages/Landing.tsx	Marketing, FAQ, hero, features
Auth	pages/auth/*, features/auth/	Login, register, Google, guards
New Report	pages/report/NewReport.tsx	Multi-step form, camera, AI gate, map
User Dashboard	pages/UserDashboard.tsx	My reports, filters, stats
Profile	pages/Profile.tsx	Name, phone, avatar, password
Public Map	pages/PublicMap.tsx	All reports on Leaflet map
Admin	pages/admin/*	Overview, reports, users, messages
Report Drawer	components/admin/ReportDrawer.tsx	Verify, assign, route preview
Priority Queue	components/admin/PriorityQueuePanel.tsx	Heap-ordered dispatch UI
API client	api/client.ts	Axios, JWT refresh, envelope unwrap

10. Backend Modules

Layer	Examples
Routes	auth.routes, report.routes, admin.routes, notification.routes,...
Controllers	Parse request, call service, JSON response
Services	auth, report, detection, admin, routing, priorityQueue, notification
Repositories	user.repo, report.repo — only Prisma access
Middleware	authenticateToken, requireRole, validateBody, upload, rate limiters
Algorithms	Isolated, testable, no HTTP dependencies

11. Security & Quality Methods

Method	Implementation
Password storage	bcrypt, never plaintext
Session	JWT access + rotating refresh, server-side refresh hash
Authorization	Role middleware on admin routes
Input validation	Zod schemas on all mutating endpoints
SQL injection	Prisma parameterized queries only
File uploads	MIME whitelist, 5 MB cap, sharp re-encode
Rate limiting	Register, login, forgot-password, reset, contact
CORS	Configurable allowlist (CORS_ORIGIN)
Headers	Helmet
Error handling	Central middleware, no stack traces to client in production
SSL to database	sslmode=require auto-appended for PostgreSQL (Supabase)

12. Deployment & Operations

Task	Command
Install	npm install (root workspace)
Dev (both apps)	npm run dev
Frontend build	npm run build -w frontend
Backend start	npm run start -w backend
DB sync	cd backend && npx prisma db push or migrate deploy
Seed demo data	npm run db:seed
Tests	npm test
Check DB	cd backend && npx tsx scripts/check-db.mjs

Environment variables (backend): see backend/.env.example — DA
CORS_ORIGIN, optional Cloudinary.
Frontend: VITE_API_URL — API base URL (defaults to /api with Vite)

See `deploy.md` for hosting notes.

13. Limitations & Future Work

13.1 AI / Detection Limitations

Limitation	Detail
Synthetic training data	CNN weights trained on generated blobs, not real pothole datasets...
Small input size	32×32 grayscale loses fine detail; good for demo/education, not...
No ML framework	Custom TS CNN is educational; production would typically use...
Scene heuristics	Skin/road rules can misclassify unusual lighting, wet roads, or non-...
Camera false positives	Live camera captures differ from uploads; scene + structure gates...
Map capture bypass	Map screenshot reports skip AI gate (<code>skipDetection</code>) by design

13.2 Routing Limitations

Limitation	Detail
Geographic coverage	OSM graph cached for Kathmandu valley only — reports far from...
Static graph	Road network not live-updated; construction closures not modeled
2 km snap limit	GPS points far from roads cannot be routed
Straight worker coords	Workers need latitude/longitude set for routing

13.3 Product / Scope Limitations

Limitation	Detail
No anonymous reporting	Account required to submit (public can only view map)
In-app notifications only	No SMS, email push, or mobile push notifications
No native mobile app	Responsive web only
Single municipality focus	Multi-tenant / multi-city not built in
English only	No i18n
No external 311 integration	No sync with municipal asset systems
Supabase dependency	Free-tier projects pause after inactivity — causes DB connection...

13.4 Technical Limitations

Limitation	Detail
Heap rebuilt on demand	Not an incremental persistent queue — fine for hundreds of report...
Duplicate radius fixed	20 m may merge distinct potholes on dense roads or miss...
Image storage	Local <code>/uploads</code> does not scale across multiple server instances...
Large frontend bundle	~1.3 MB JS — code-splitting recommended for production...
Prisma migrate history	Provider switch (SQLite !" PostgreSQL) requires careful migration...

13.5 Recommended Improvements

1. Train CNN on real labeled pothole dataset (e.g. Pothole-600, RDD2022) 2. Replace or augment CNN with YOLOv8 / ONNX Runtime 3. Expand OSM graph fetch to deploy region or use routing API (OSRM, GraphHopper) 4. Add email notifications for status changes 5. Redis-backed priority queue for scale 6. Multi-region admin tenancy 7. Progressive Web App for offline draft reports

Related Documentation

Document	Link
Software requirements	srs.md
Algorithm deep dive	algorithms.md
API reference	api.md
Architecture	architecture.md
Database	database.md
Diagrams (Mermaid)	diagrams.md
Deployment	deploy.md